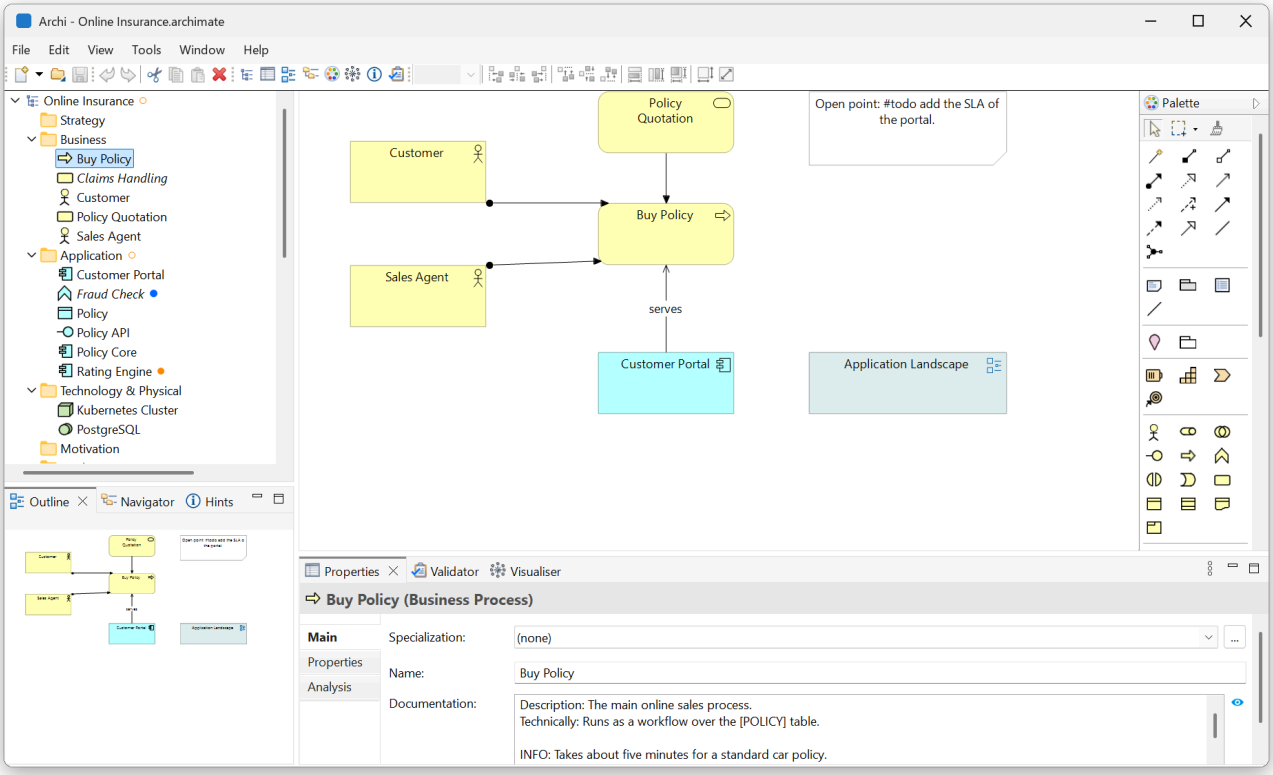
documentationArchi – User Guide

Usage guide for the Archi plugin

documentationArchi is a plugin for Archi that turns an ArchiMate model into documentation people can read without Archi: **HTML pages**, a **PDF document** and **Confluence pages**. Every view gets its own page with its picture and a description of each element on it, built from the descriptions, properties and relationships you already keep in the model.

This guide explains every function of the plugin: where to find it, what it needs from you and what it produces.



Contents

1. At a glance
2. Installation
3. Quick start
4. Where the functions are
5. Export Documentation
6. The exported documentation
7. Print Order
8. Work packages (Jira tickets)
9. Change marks in the Models tree
10. Formatting descriptions
11. Documentation Formatting (changing the look)
12. Assets: pictures and files
13. Publishing to Confluence
14. Command line and automation
15. Where things are stored
16. Troubleshooting

1. At a glance

Function	Where	What it takes	What it gives
Export Documentation...	Right-click in the Models tree or in a view	A model, folder, views, elements, relationships or work packages; a target folder; the formats	HTML pages, a PDF and Confluence pages in a folder named after the model
Print Order...	Right-click a view (tree or view editor)	A view	The order in which the export describes the view's elements, saved in the model
Link to Work Package...	Right-click elements	One or more elements	Links between the elements and Jira-ticket work packages
Work Packages...	Right-click anything in a model	A model	An overview of all work packages, what they change and what isn't exported yet
Add Asset...	Right-click one object	One element, relationship, view, note or the model; files from disk	The files copied into `Assets`, and a link to them in the description
Change marks	Models tree (automatic)	Nothing – it watches the model	Dots showing what no export reflects yet
Documentation Formatting...	**Tools** menu, **Preferences > Documentation Export**	Your choice of colours, sizes and keywords	How headings, tokens, SQL, labels and panels look in every export
Command line export	Archi's command line	A model file and a folder	The same documentation, for CI and scheduled builds

2. Installation

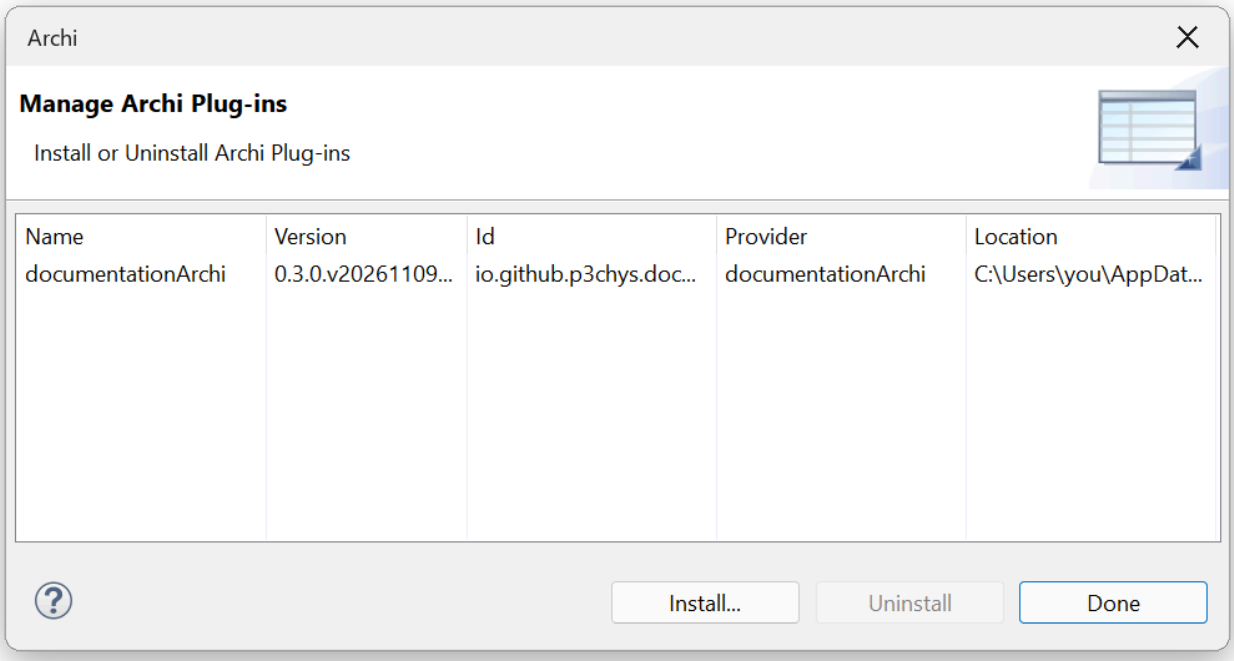
Requirements

- **Archi 4.9 or later**, on Windows or macOS. One plugin file works with every supported Archi version.
- To publish to Confluence: **Python 3.8 or later** on the machine that uploads (no extra packages needed).

Archi	Tested in
4.9.x	4.9.3
5.0 – 5.3	5.1.0, 5.3.0
5.4 – 5.9	5.4.3, 5.9.0
5.10 and up	5.10.0

Install

1. Download `documentationArchi-<version>.archiplugin` from the releases page.
2. Start Archi and choose **Help > Manage Plug-ins...**.
3. Click **Install...**, select the downloaded `.archiplugin` file and confirm.
4. Restart Archi when asked.



Check that it works

After the restart:

- right-click the model in the **Models** tree – the menu ends with **Export Documentation...**;
- the **Tools** menu has **Documentation Formatting...**;
- the preferences (**Edit > Preferences...**; on macOS in the **Archi** application menu) have a **Documentation Export** page.

Upgrade

Open **Help > Manage Plug-ins...**, select **documentationArchi**, click **Uninstall**, then install the new file as above and restart. Your formatting settings, export folder and change marks are kept.

Uninstall

Help > Manage Plug-ins..., select **documentationArchi**, **Uninstall**, restart Archi. Models stay as they are: print orders and work-package links are ordinary ArchiMate elements and relationships.

> **Windows tip:** don't start Archi from *PowerShell 7 installed from the Microsoft Store*. Store apps run in a sandbox that redirects `%APPDATA%` (where Archi keeps its plug-ins and settings), so Archi would not see the plugin. Start Archi normally, or from cmd, Windows PowerShell or a winget/MSI install of PowerShell 7.

3. Quick start

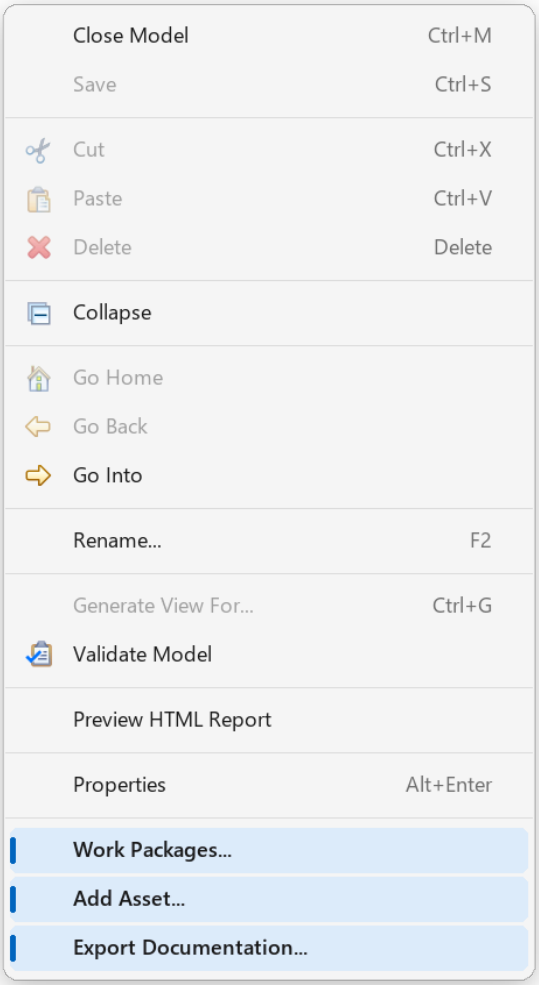
1. Open a model and save it (the plugin looks for assets next to the `.archimate` file).
2. Right-click the model in the **Models** tree and choose **Export Documentation...**.
3. Pick a folder, keep **HTML**, **PDF** and **Confluence** ticked, and click **Export**.
4. When the export finishes, click **Open Folder** and open `html/index.html` or the PDF.

Everything else in this guide refines that result: the order of elements, Jira-ticket exports, formatting, pictures and publishing.

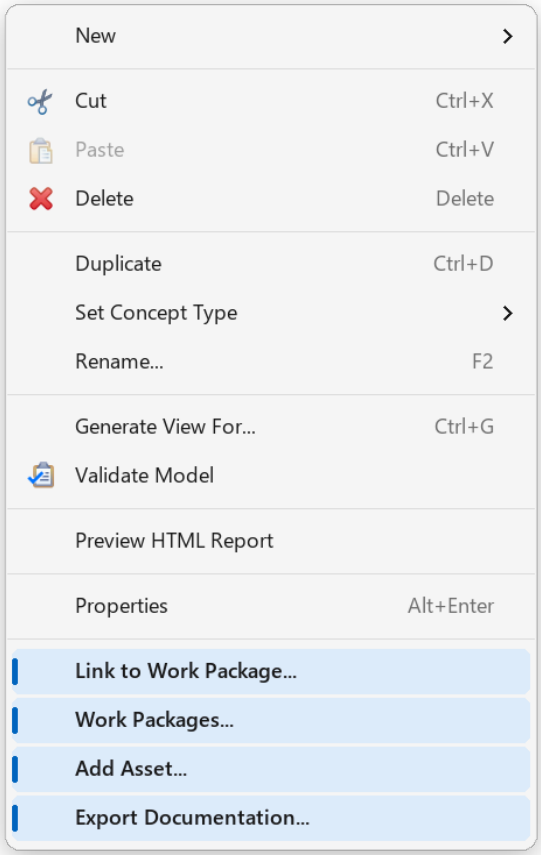
4. Where the functions are

The plugin adds its commands to the end of the context menus of the **Models** tree and of the view editors, to the **Tools** menu, and to the preferences. Its entries are highlighted in the menus below; they appear only when the selection fits.

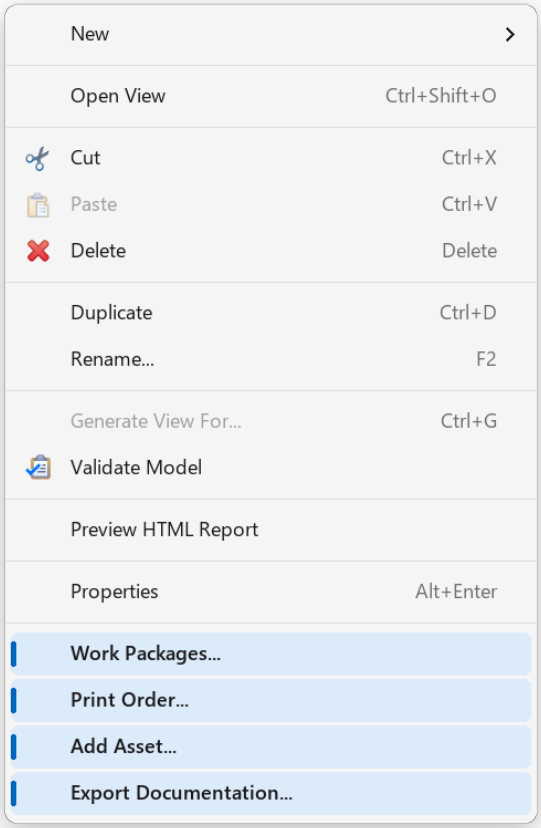
Models tree – right-click the model



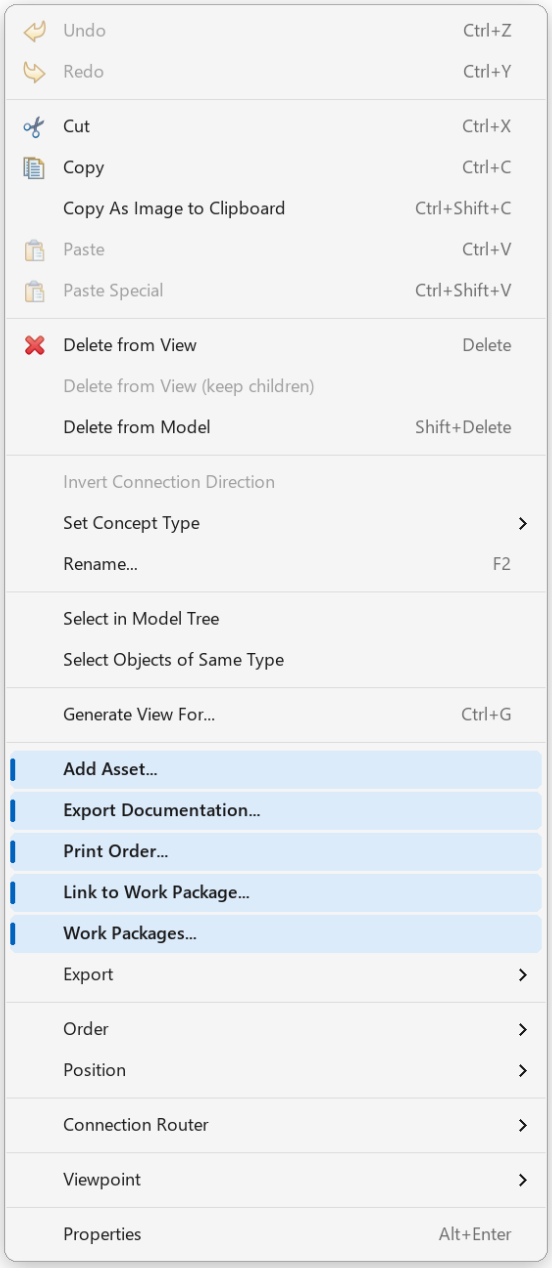
Models tree – right-click an element



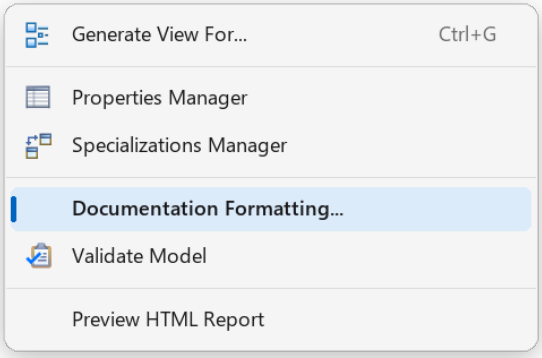
Models tree – right-click a view



View editor – right-click an element



Tools menu



Command	Models tree	ArchiMate view	Sketch / Canvas view	Shown when
Export Documentation...	✓	✓	✓	anything in a model is selected
Add Asset...	✓	✓	✓	exactly one element, relationship, view, note or the model
Print Order...	✓	✓	–	one view (tree), or objects of the view (editor)
Link to Work Package...	✓	✓	–	the selection holds elements
Work Packages...	✓	✓	–	anything in a model is selected

Command	Models tree	ArchiMate view	Sketch / Canvas view	Shown when
Documentation Formatting...	**Tools** menu			always

5. Export Documentation

Produces the documentation of whatever you right-clicked.

What it takes

- A selection in the Models tree or in a view (see the table below).
- A target folder and one or more formats.

What it gives

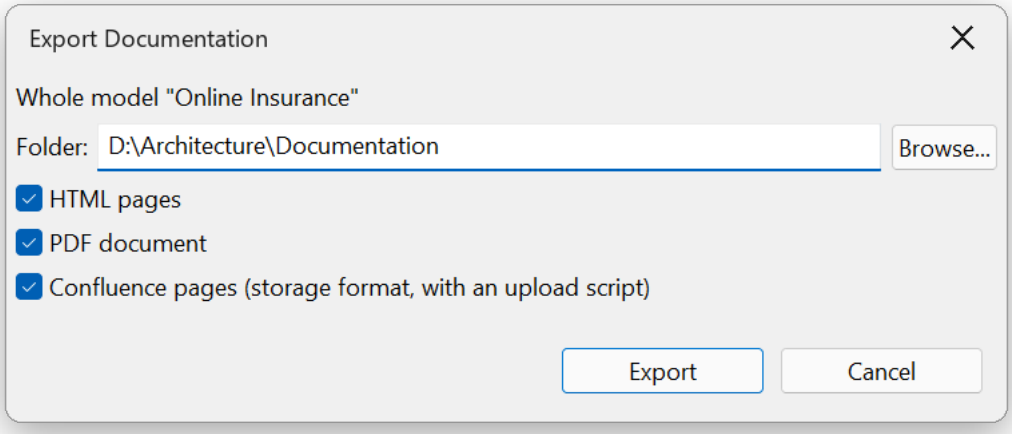
- A folder named after the model (or the work package) inside the target folder, holding the HTML pages, the PDF and the Confluence pages.

What gets exported

What you right-click decides the content:

You select	The export contains
the model	every view, an overview page, a page for elements that are in no view, and a *Work packages* page
work packages	only the elements they change (details)
a folder	everything in it
views	those views
elements	a page *Selected elements* describing them
a relationship	the elements at both ends
objects in a view	their elements; a note or group means its whole view

The dialog



- The first line says what will be exported, for example *Whole model "Online Insurance"* or *2 view(s) and 3 element(s) of "Online Insurance"*.
- **Folder** – where the export goes; **Browse...** picks it. The export itself goes into a subfolder named after the model, for example `online-insurance`.
- **HTML pages**, **PDF document**, **Confluence pages** – the formats. **Export** stays disabled until at least one is ticked.

The folder and the formats are remembered for the next export. If the subfolder already holds files, you're asked before they are replaced.

While and after it runs

The export runs as a background job (*Exporting documentation*), so you can keep working. When it finishes, a message says *Exported to <folder>* and offers **Open Folder** and **Close**. If a description refers to an asset that doesn't exist, the message lists those references (up to 15) and the pages show them in red – see Assets.

The export folder

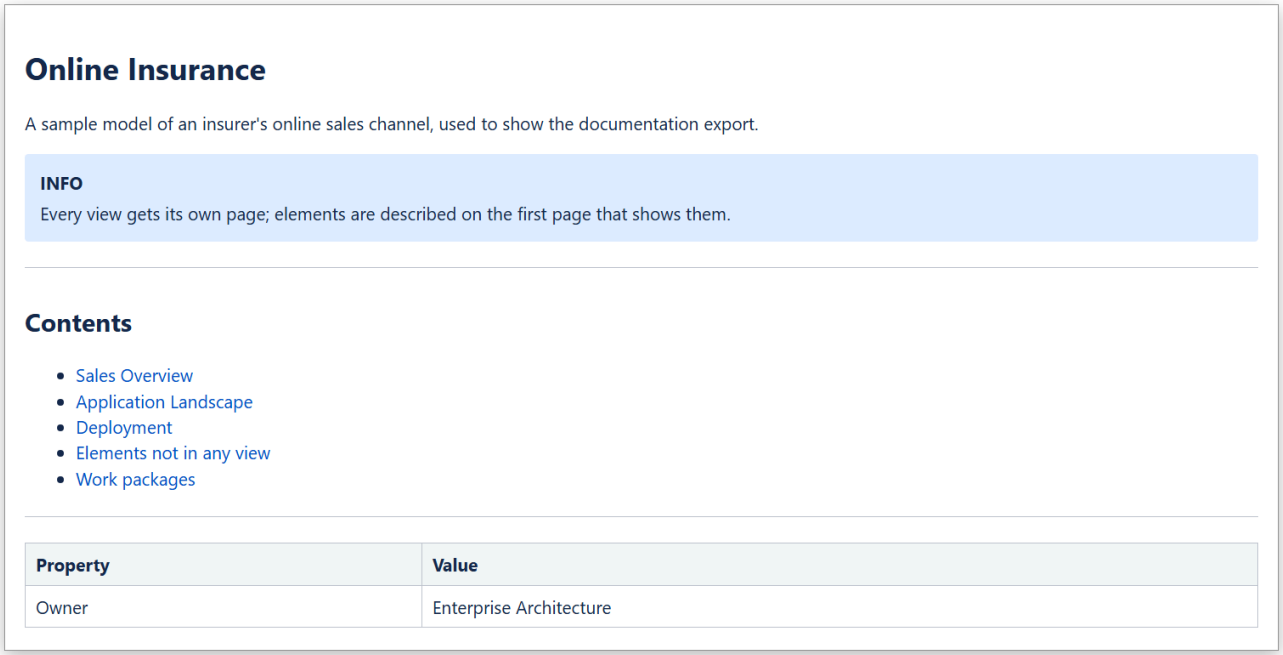
```
online-insurance/
├── html/
│   ├── index.html                overview page – open this one
│   ├── sales-overview.html       one page per view
│   ├── elements-not-in-any-view.html
│   ├── work-packages.html
│   ├── images/                   view pictures
│   ├── assets/                   copies of the assets the pages use
│   └── style.css
├── online-insurance.pdf          the whole documentation as one PDF, with bookmarks
├── assets/                       files the PDF links to
└── confluence/
    ├── pages/                    pages in Confluence storage format
    ├── attachments/              view pictures and assets
    ├── manifest.json
    └── upload_to_confluence.py   the upload script
```

Every export carries its own copies of pictures and files, so you can zip it, mail it or move it without breaking links.

6. The exported documentation

Overview page

With more than one page, the export starts with an overview: the model's purpose (its description), its properties and the list of pages.



Online Insurance

A sample model of an insurer's online sales channel, used to show the documentation export.

INFO
Every view gets its own page; elements are described on the first page that shows them.

Contents

- Sales Overview
- Application Landscape
- Deployment
- Elements not in any view
- Work packages

Property	Value
Owner	Enterprise Architecture

View pages

Each view gets a page with, in this order:

1. the view's description,
2. *Contents* – the elements of the view with their types,
3. the view's picture (drawn at twice the size, so it stays sharp in the PDF and when zooming in),
4. the view's properties,
5. *Notes* – the text of the notes on the view,
6. *Related views* – views this view refers to,
7. *Elements* – each element described in full.

Online Insurance

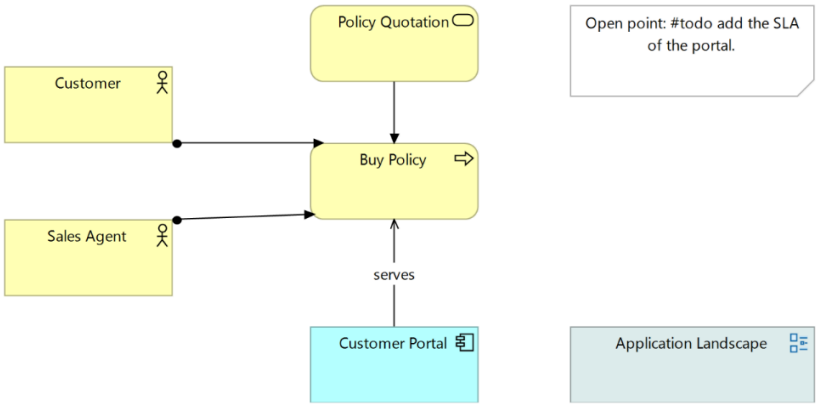
Sales Overview

How a customer buys a policy online.

NOTE
This view reflects the 2026.4 release.

Contents

- <<Business Process>> Buy Policy
- <<Business Actor>> Customer
- <<Application Component>> Customer Portal
- <<Business Service>> Policy Quotation
- <<Business Actor>> Sales Agent



Element descriptions

Each element is described with its description, its properties, a table of its relationships to other elements of the view (with each relationship's own description), the views it appears on, its work packages, and its relationships to elements outside the view, grouped by type.

<<Business Process>> **Buy Policy**

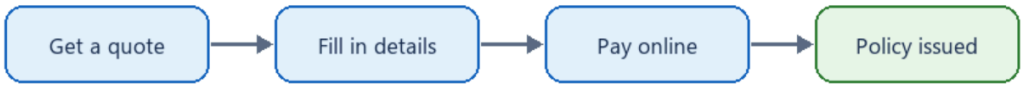
Description: The main online sales process.
Technically: Runs as a workflow over the [POLICY] table.

INFO
Takes about five minutes for a standard car policy.

Outage risk RISK, the paper channel is DEPRECATED.

Reads the policies of customer {customer_id} with status 'ACTIVE' from [POLICY].

```
select p.policy_no, p.status
  from policy p
 where p.status = 'ACTIVE'
   and p.customer_id = :customer_id
```



Process flow:

Property	Value
Owner	Sales
SLA	99.5 %

Relationship	Element	Description
← Assignment Relationship	<<Business Actor>> Customer	
← Assignment Relationship	<<Business Actor>> Sales Agent	
← Serving Relationship serves	<<Application Component>> Customer Portal	**WARNING** The only sales channel outside office hours.
← Triggering Relationship	<<Business Service>> Policy Quotation	

Views: Sales Overview

- An element is described **once**, on the first page that shows it. Later pages link to that description. Links work in all three formats.
- Elements are described in alphabetical order unless the view has a print order.

The PDF

The PDF holds the same pages, with bookmarks for every page and page numbers. Large pictures are scaled to fit the page.

Online Insurance

A sample model of an insurer's online sales channel, used to show the documentation export.

INFO
Every view gets its own page; elements are described on the first page that shows them.

Contents

- Sales Overview
- Application Landscape
- Deployment
- Elements not in any view
- Work packages

Property	Value
Owner	Enterprise Architecture

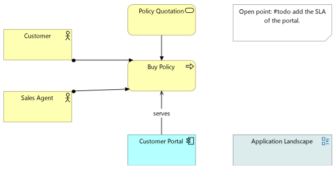
Sales Overview

How a customer buys a policy online.

NOTE
This view reflects the 2026.4 release.

Contents

- <<Business Process>> Buy Policy
- <<Business Actor>> Customer
- <<Application Component>> Customer Portal
- <<Business Service>> Policy Quotation
- <<Business Actor>> Sales Agent



Property	Value
Version	1.2

Notes

Open point: **TODO** add the SLA of the portal.

Related views

- Application Landscape

Elements

<<Business Process>> **Buy Policy**

Description: The main online sales process.
Technically: Runs as a workflow over the [POLICY] table.

INFO
Takes about five minutes for a standard car policy.

Outage risk **RISK**, the paper channel is **DEPRECATED**.

Reads the policies of customer {customer_id} with status 'ACTIVE' from [POLICY].

```
select p.policy_no, p.status
  from policy p
 where p.status = 'ACTIVE'
   and p.customer_id = :customer_id
```

Process flow:



Property	Value
Owner	Sales
SLA	99.5 %

Relationship	Element	Description
← Assignment Relationship	<<Business Actor>> Customer	
← Assignment Relationship	<<Business Actor>> Sales Agent	
← Serving Relationship serves	<<Application Component>> Customer Portal	**WARNING** The only sales channel outside office hours.
← Triggering Relationship	<<Business Service>> Policy Quotation	

Views: Sales Overview

<<Business Actor>> **Customer**

A person who buys and renews insurance policies online.

WARNING
Customer data is personal data under GDPR.
Only authorised roles may read it.

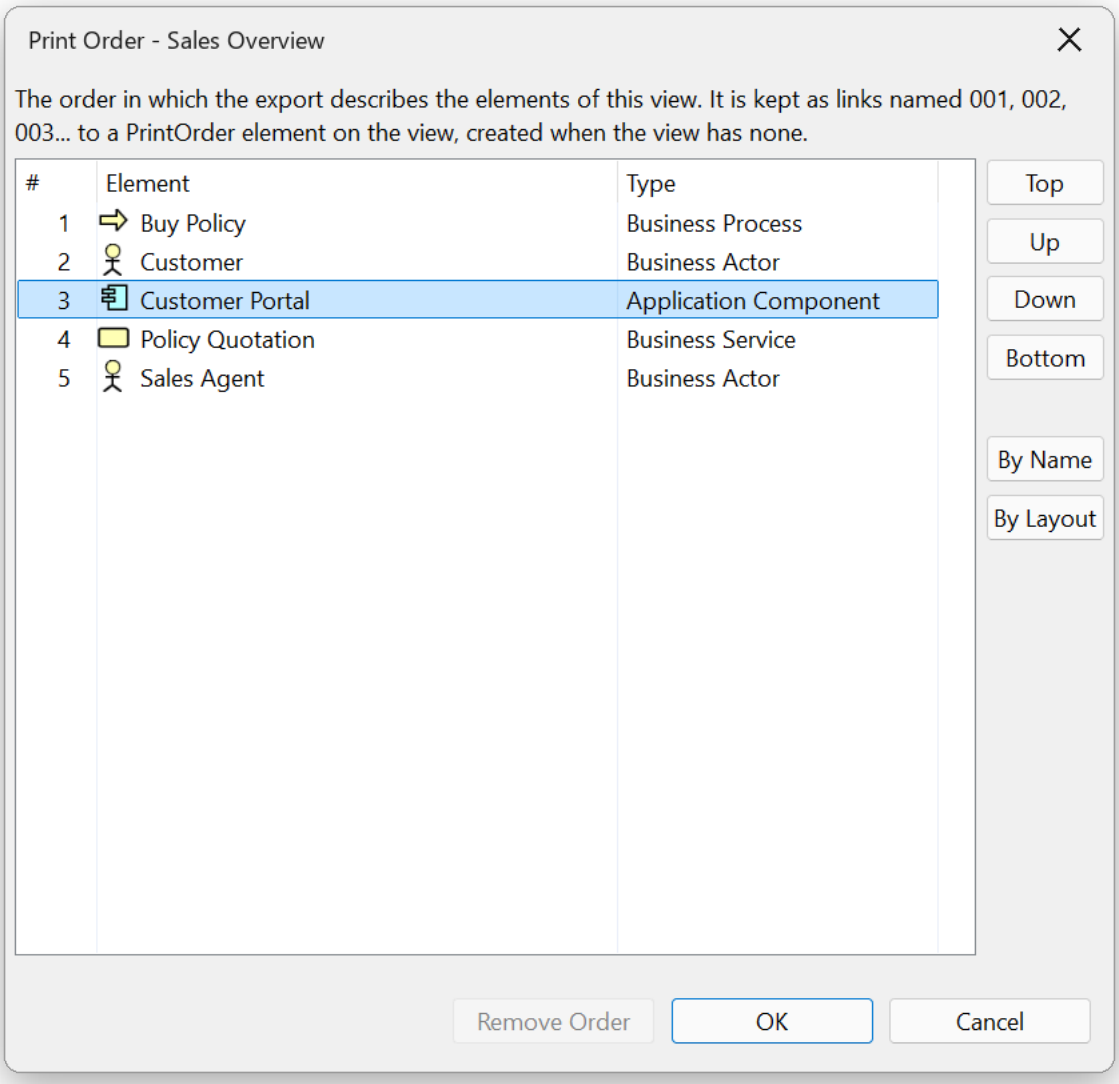
Status: **DRAFT**, see https://example.com/privacy-policy.

7. Print Order

Sets the order in which the export describes the elements of a view, instead of alphabetical.

What it takes: one ArchiMate view – right-click it in the Models tree, or right-click anything on the view in its editor.

What it gives: the chosen order, saved in the model so everyone who exports the model gets the same order.



Control	Does
Top, **Up**, **Down**, **Bottom**	move the selected elements; several can be moved together
Alt+Up / **Alt+Down**	the same from the keyboard
drag and drop	moves the selected rows to where you drop them
By Name	sorts by name

Control	Does
By Layout	sorts as the elements appear on the view: top to bottom, then left to right
Remove Order	takes the view's order away, back to alphabetical (enabled only when the view has an order)
OK	saves the order

How it's stored: a Grouping named `PrintOrder` on the view, with associations named `001`, `002`, `003`… from each element to it – the way many modellers do it by hand, so you can also draw them yourself. Elements linked to a PrintOrder come first, by the number in the link's name; the rest follow alphabetically. The PrintOrder element and its links never appear in the export or its pictures. **Edit > Undo** takes a change back.

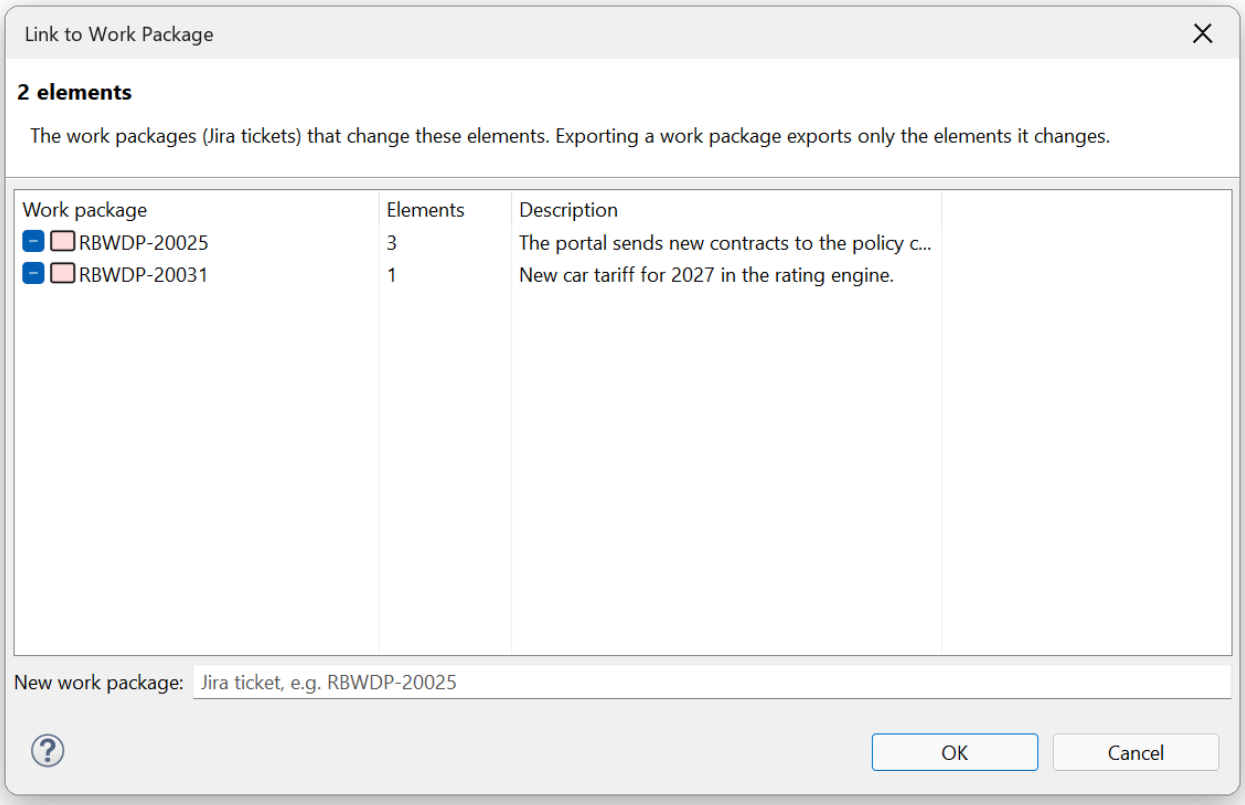
8. Work packages (Jira tickets)

A work package is an ArchiMate **Work Package** element named after a Jira ticket, such as `RBWDP-20025`. Linking an element to it says *this element changes in that ticket*. Exporting the work package gives developers a specification of just those elements.

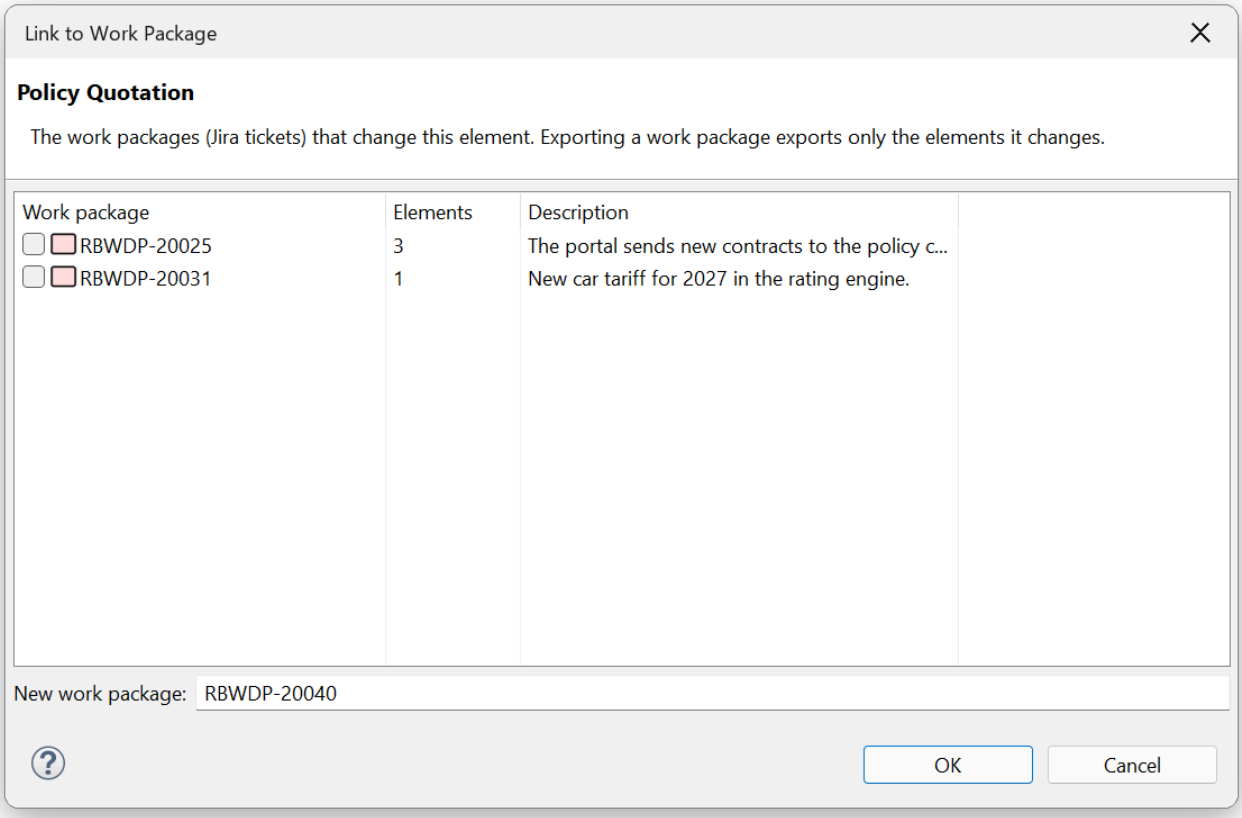
Link to Work Package

What it takes: one or more elements (in the Models tree or in a view).

What it gives: association relationships between the elements and the work packages you tick.



- The table lists every work package of the model, how many elements it changes and the first line of its description.
- **Tick** a work package to link all selected elements to it; **untick** it to take them off.
- A **grey tick** (shown as **–**) means only some of the selected elements are linked. It stays as it is unless you click it, which links all of them.
- **New work package** creates one and links the selected elements to it. It goes into the *Implementation & Migration* folder. The name must start with the ticket key – capital letters, a dash and a number, such as `RBWDP-20040` – and may go on with a title; otherwise the dialog says so and **OK** is disabled. Typing the name of a work package that already exists links to that one.

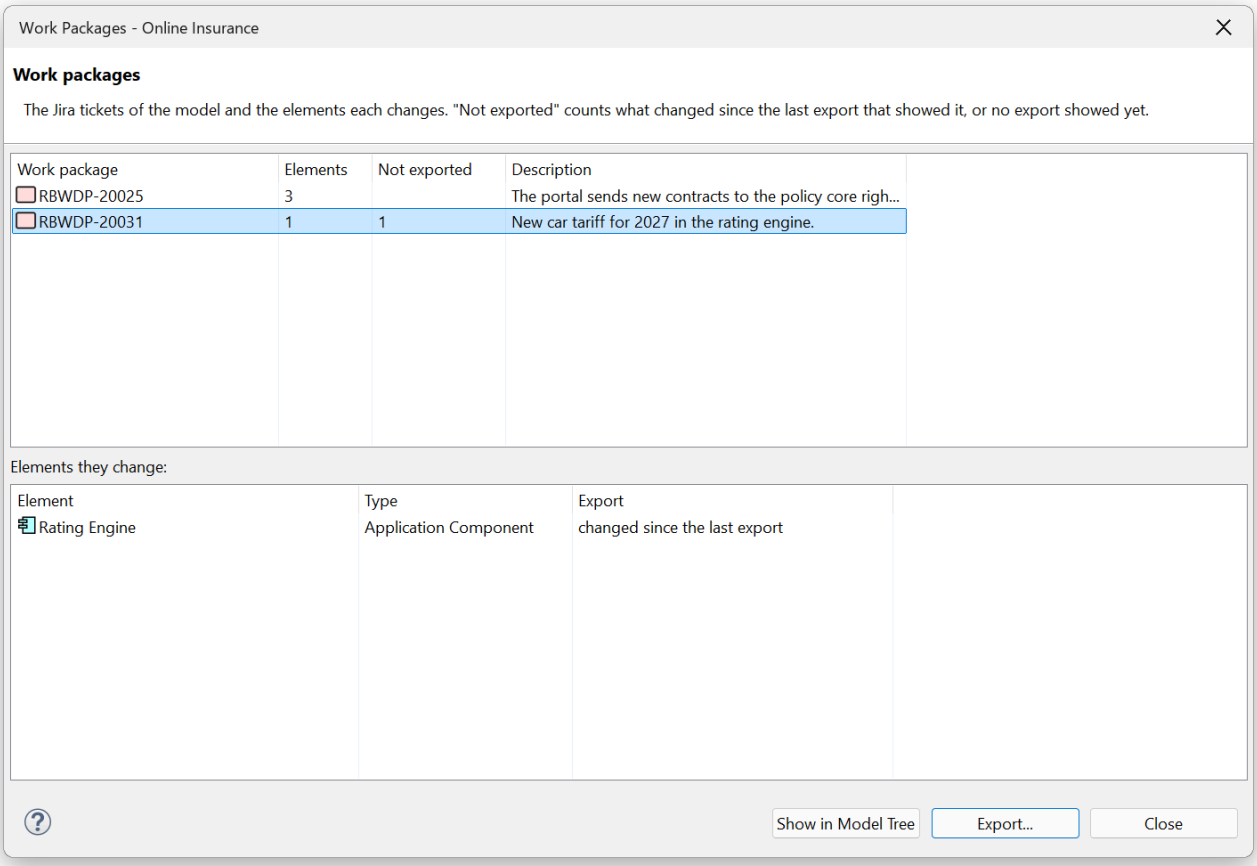


Edit > Undo takes the whole change back. The links are plain associations in either direction, so you can draw them yourself too. An element can be in several work packages.

Work Packages overview

What it takes: a right-click on anything in the model.

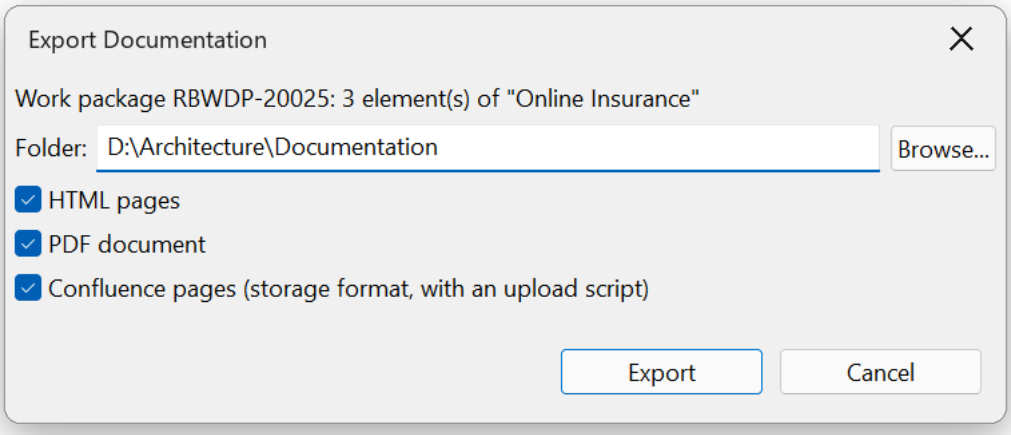
What it gives: every work package, what it changes and what no export shows as it is now – with export and navigation from there.



- **Elements** – how many elements the work package changes.
- **Not exported** – how many of them changed since the last export that showed them, or were never exported (see change marks).
- Select one or more work packages to list their elements below. The **Export** column says *in no export yet*, *changed since the last export* or *newly linked since the last export*.
- **Export...** (or a double-click) exports the selected work packages.
- **Show in Model Tree** selects the chosen elements – or, if none are chosen, the work packages – in the Models tree.

Export a work package

Right-click one or more work packages and choose **Export Documentation...**, or use **Export...** in the overview.



The export holds:

- a first page named after the ticket, with the work package's description and properties (the specification) and the elements it changes;
- the views showing any of these elements – each with its full picture, but describing only those elements;
- the elements that are on no view.

In every export, an element's description ends with the work packages that change it, and a whole-model export has a *Work packages* page describing each ticket:

Work packages

Contents

- <<Work Package>> RBWDP-20025
- <<Work Package>> RBWDP-20031

Elements

<<Work Package>> **RBWDP-20025**

The portal sends new contracts to the policy core right after the purchase.

TODO: agree on the message format with the core team.

Property	Value
Release	2027.1

Changed elements

- <<Application Component>> Customer Portal
- <<Application Interface>> Policy API
- <<Application Component>> Policy Core

<<Work Package>> **RBWDP-20031**

New car tariff for 2027 in the rating engine.

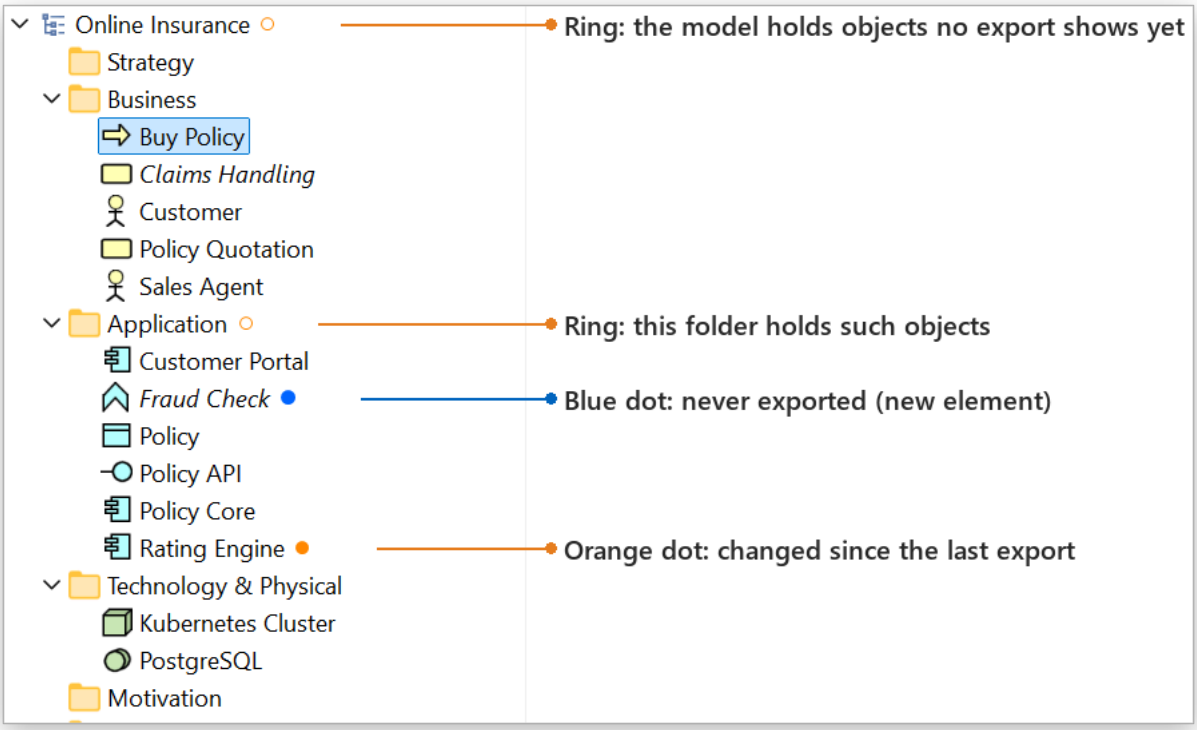
Property	Value
Release	2027.1

Changed elements

- <<Application Component>> Rating Engine

9. Change marks in the Models tree

The Models tree shows, with a small mark after the name, what no documentation export reflects yet – so you know what to export before you hand documentation over.



Mark	Means
orange dot	the element, relationship or view changed since the last export that showed it
blue dot	no export has shown it yet (new, or never exported)
orange / blue ring	a folder (or the model) holding such objects, so you find them with the tree collapsed

- Hover over a mark to see what it means and when the object was last exported.
- **Any** export counts: exporting a work package brings its elements up to date.
- A view counts as changed when anything on it moves or changes, or an element on it is renamed, because its picture changes.
- PrintOrder elements and their links never get a mark.
- To hide the marks, untick **Mark in the model tree what no export shows yet** in Preferences > Documentation Export.

10. Formatting descriptions

Write these markers in any description (of the model, a view, an element or a relationship), in notes and in property values. They look the same in HTML, the PDF and Confluence (where they become Confluence's own panels, status labels and code blocks).

Write	Result
`INFO: text`, `NOTE: text`, `TIP: text`, `WARNING: text` at the start of a line	a coloured panel with that title, up to the next blank line
`<#>a remark</#>` (or up to the next blank line)	a note panel without a title
`#risk` or `[[risk]]`	a label; built in: `risk` (red), `deprecated` (grey), `todo` and `draft` (yellow), `new` (green)
`Description:`, `Technically:` anywhere	bold
`[ORDERS]`	a table name, coloured
`'ACTIVE'`	a constant, coloured
`{client_id}`	a variable, coloured
`<sql>select ...</sql>`	an SQL code block
`https://...`	a link
`[[asset:file.png]]`	a picture or file from the Assets folder – see Assets
a blank line	a new paragraph; single line breaks stay

Keywords ignore case. Labels and panels only work for configured keywords, so other `#words` stay as they are. You can add your own keywords – see the next section.

Example. This description of *Buy Policy*:

```
Description: The main online sales process.
Technically: Runs as a workflow over the [POLICY] table.

INFO: Takes about five minutes for a standard car policy.

Outage risk #risk, the paper channel is [[deprecated]].

Reads the policies of customer {customer_id} with status 'ACTIVE' from [POLICY].

<sql>
select p.policy_no, p.status
  from policy p
 where p.status = 'ACTIVE'
   and p.customer_id = :customer_id
</sql>

Process flow: [[asset:process/buy-policy.png|Buy Policy process flow]]
```

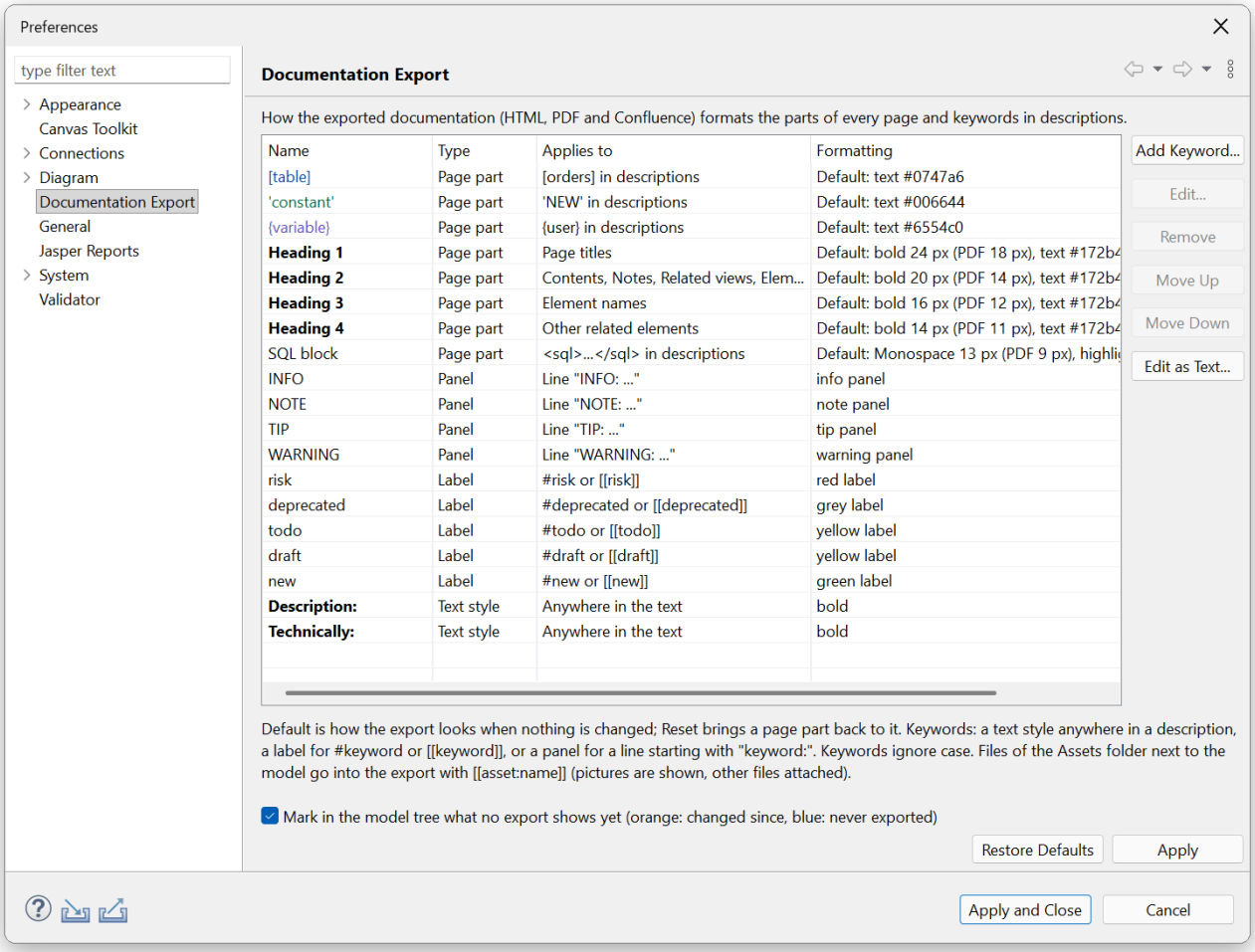
is exported as shown in Element descriptions.

11. Documentation Formatting (changing the look)

Where: Tools > Documentation Formatting..., or **Preferences > Documentation Export** (the same page, shown inside the preferences).

What it takes: your choice of colours, sizes, effects and keywords.

What it gives: the look of every later export, in all three formats. The settings are kept in Archi's preferences on your computer, not in the model.

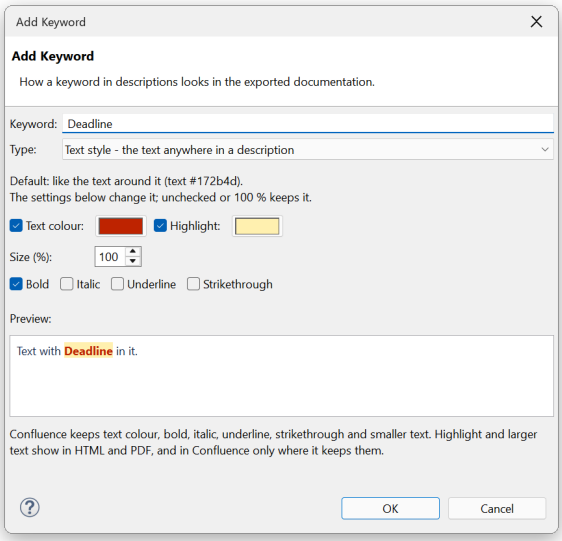


The list holds everything whose look can change:

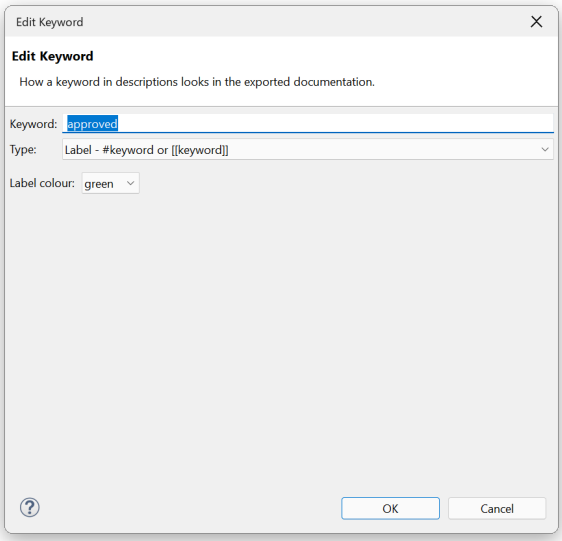
- **Page parts** – the `[table]`, `'constant'` and `{variable}` tokens, headings 1–4 and SQL blocks. They are always there; **Edit...** changes one and **Reset** (the *Remove* button, when page parts are selected) brings it back to the default shown in the *Formatting* column.
- **Keywords** – panels, labels and text styles. **Add Keyword...**, **Edit...**, **Remove**, **Move Up** and **Move Down** manage them. Double-click a row to edit it.
- **Edit as Text...** – all rules as text, see below.
- **Mark in the model tree what no export shows yet** – switches the change marks on or off.
- **Restore Defaults** brings back the built-in keywords and page-part looks.

Add or edit a keyword

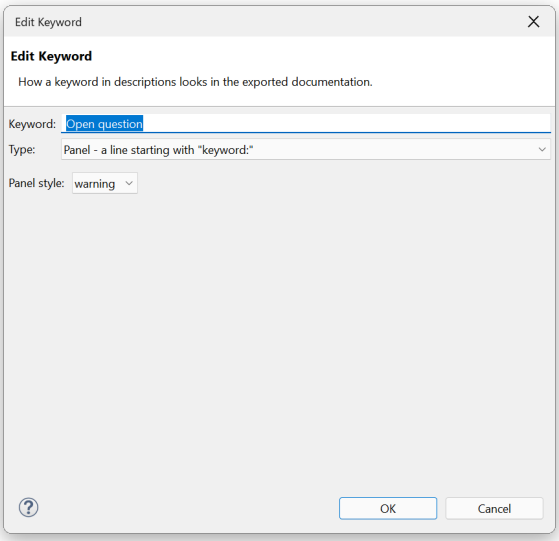
Choose the **Type** first; the options below it change with it. Every editor shows a live preview.



Text style – this text always gets a colour, highlight, size, bold, italic, underline or strikethrough



Label – `#keyword` or `[[keyword]]` becomes a grey, red, yellow, green or blue label

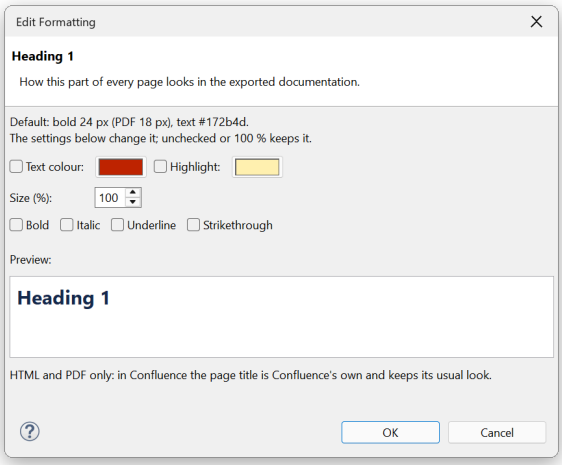


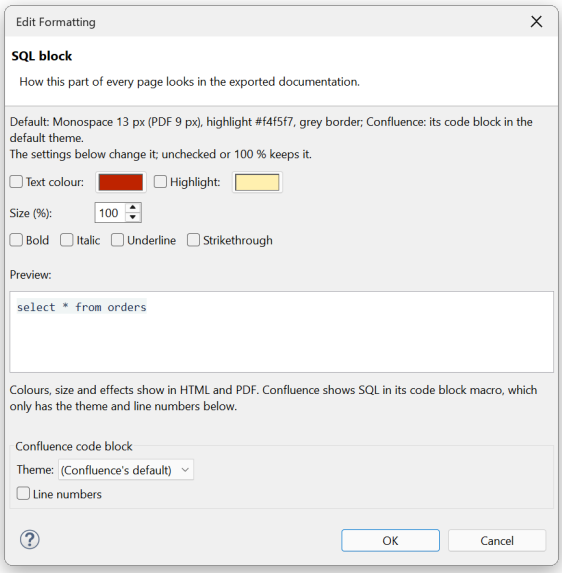
Panel – a line starting with `keyword:` becomes an info, note, tip or warning panel

- A panel keyword is at most 40 characters and has no `:`; a label keyword cannot contain `]`.
- A keyword of the same type and name as an existing one replaces it.

Edit a page part

Selecting a page part and clicking **Edit...** opens the same style editor, starting from the default look. For **SQL block** it also sets Confluence's code block **Theme** and **Line numbers**.

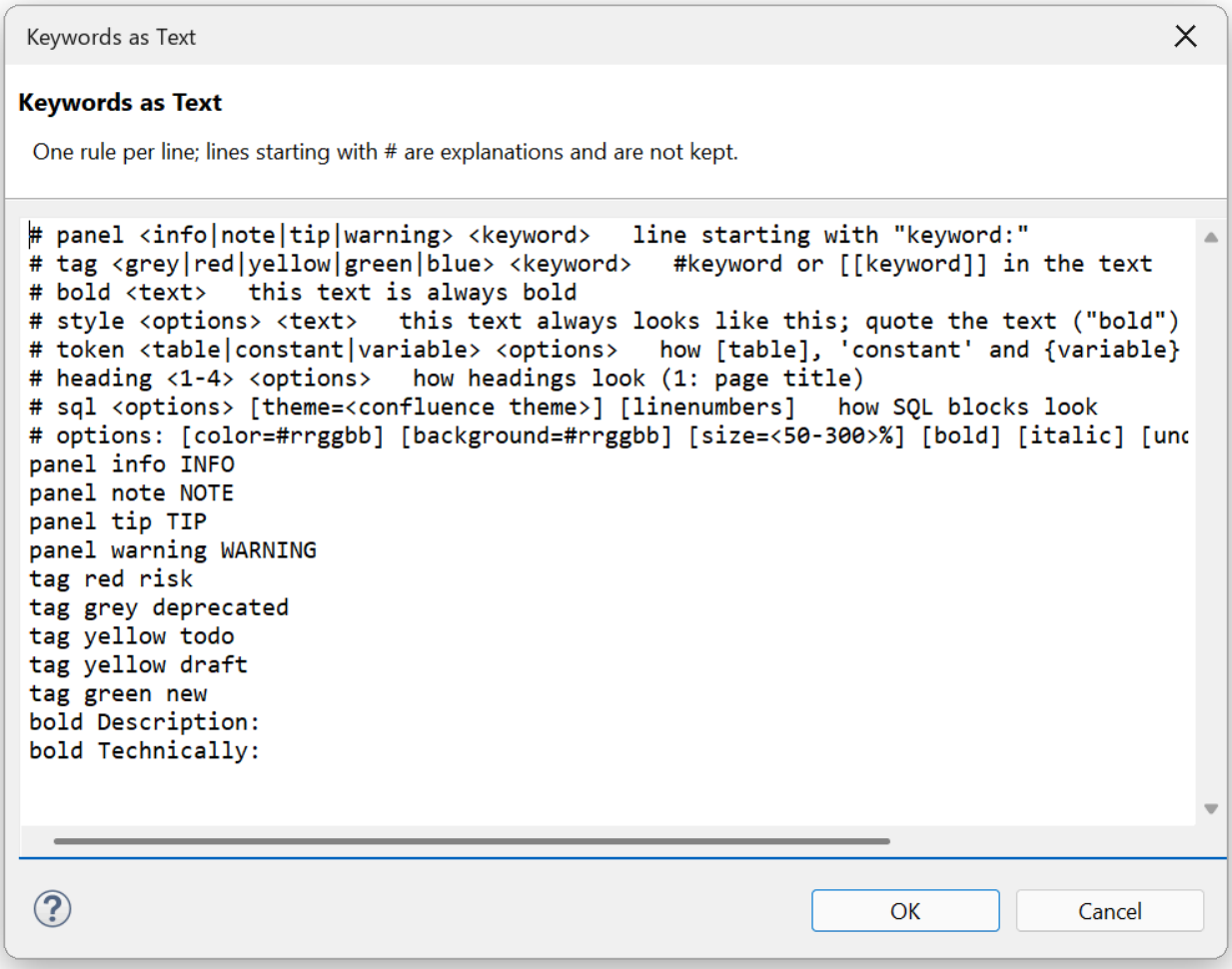




Confluence keeps text colour, bold, italic, underline, strikethrough and smaller text. It doesn't keep every background or larger size in every version; the editors say so where it matters.

Edit as Text

Shows all rules as text, one per line – to copy them to another computer or change many at once. Lines starting with `#` are explanations and are not kept. An invalid line is reported at the top and **OK** stays disabled until it's fixed.




```
panel warning Open question           a line "Open question: ..." becomes a warning panel
tag red risk                          #risk or [[risk]] becomes a red label
bold Description:                     this text is always bold
style color=#bf2600 background=#ffebe6 size=120% bold italic Deadline
token variable color=#6554c0 italic   how {variable} looks; also table and constant
heading 2 color=#0052cc size=120%     how headings of a level (1-4) look
sql background=#f4f5f7 theme=midnight linenumbers
```

12. Assets: pictures and files

Keep pictures and documents in a folder named `Assets` next to the `.archimate` file (subfolders are fine) and put them into the documentation from any description, note or property value.

Add Asset

What it takes: exactly one element, relationship, view, note or the model (right-click it in the Models tree or in a view), and one or more files you pick. The model must have been saved, so the plugin knows where `Assets` is.

What it gives:

- the files copied into `Assets` (created if missing). A file already inside `Assets` is used where it is; the same file picked again is reused; a different file with the same name is saved with `-2`;
- one `[[asset:...]]` line per file at the end of the description.

Edit > Undo removes the lines; the copied files stay.

Referring to assets by hand

Write	Result
`[[asset:process/buy-policy.png]]`	the picture, shown in the text
`[[asset:process/buy-policy.png|Buy Policy process flow]]`	the same, with that alternative text
`[[asset:contracts/SLA 2026.pdf|the SLA]]`	a link "the SLA" that opens the file
`asset:tariff.csv`	short form, for names without spaces

- A file name alone is enough when only one file in `Assets` has it; otherwise use its path inside `Assets`. Case and `/` or `\` don't matter.
- PNG, JPEG, GIF, BMP and SVG are pictures; anything else is a file to open.
- Large pictures are shown smaller, keeping their proportions: in the PDF never wider or taller than a page; on HTML and Confluence pages at most about 860 pixels high – click to see full size.

How each format carries them

Format	Assets
HTML	copied to `html/assets/`
PDF	pictures are drawn into the PDF; other files (and SVG) are links to the copy in `assets/` next to the PDF, and are also attached inside the PDF
Confluence	attachments of the page

A reference that finds no file shows in red as *[missing asset: name]* and is listed when the export finishes. Only files really inside `Assets` are used: links pointing outside it are ignored and reported, so a model shared with you can't pull other files from your computer into an export.

13. Publishing to Confluence

The Confluence export contains ready-made pages in Confluence's storage format – links, panels, labels, code blocks, pictures and assets as attachments – plus a script that uploads them to **Confluence Data Center / Server**.

What it takes: Python 3.8+, the Confluence URL, a space key and a personal access token (or user name and password).

What it gives: one Confluence page per exported page, with its attachments.

```
CONFLUENCE_TOKEN=<personal access token> python online-insurance/confluence/upload_to_confluence.py
\
  --url https://confluence.example.com --space ARCH
```

Option / variable	Meaning
`--url`	Confluence base URL (required)
`--space`	space key (required)
`--parent "Existing page"`	put the pages under an existing page
`CONFLUENCE_TOKEN`	personal access token
`CONFLUENCE_USER`, `CONFLUENCE_PASSWORD`	used when there is no token

Pages are matched by title, and existing ones get a new version, so you can upload the same export again after changes. Page titles start with the model name, because Confluence titles must be unique in a space.

14. Command line and automation

For CI pipelines and scheduled documentation builds, the plugin adds an option to Archi's command line:

```
Archi -application com.archimatetool.commandline.app -consoleLog -nosplash \
  --loadModel model.archimate --documentation.folder out [--documentation.formats
html,pdf,confluence]
```

Option	Meaning
`--documentation.folder`	export the whole model's documentation to this folder
`--documentation.formats`	comma separated `html`, `pdf`, `confluence`; default: all three

The command line exports the whole model and uses the formatting settings of the Archi installation it runs in. Missing assets are logged as errors.

15. Where things are stored

What	Where	Shared with others?
Print orders	in the model (PrintOrder grouping and links)	yes, with the model
Work packages and their links	in the model (Work Package elements, associations)	yes, with the model
Asset files	the `Assets` folder next to the model file	yes, if you share the folder
Formatting and keywords	Archi's preferences on your computer	no – use **Edit as Text...** to copy them
Export folder and formats	Archi's preferences on your computer	no
What each export showed (change marks)	Archi's settings folder, per model, per computer	no – exporting never changes the model

16. Troubleshooting

Problem	Solution
The menu entries don't appear	Restart Archi after installing. Check **Help > Manage Plug-ins...** lists documentationArchi. On Windows, don't start Archi from PowerShell 7 from the Microsoft Store (see Installation).
Save the model first: its assets are kept in the Assets folder next to the model file.	**Add Asset...** needs the model saved to a file. Save it and try again.
The work package ... changes no elements yet.	Link elements to it with **Link to Work Package...** first.
Select the elements a work package changes (work packages themselves can't be linked).	Select ordinary elements, not work packages.
The selection spans several models.	Select objects of one model only.
The view shows no elements to order.	The view holds only notes, groups or references; there is nothing for the print order.
A picture shows as *[missing asset: name]* in red	The file isn't in `Assets`, the name matches several files (use its path), or it links outside `Assets`. The export's final message lists each case.
The PDF can't show an SVG	SVG files become links to the copy in `assets/` next to the PDF. Use PNG for pictures that must appear in the PDF.
Old plugin still active after an upgrade	Uninstall the old version in **Manage Plug-ins** before installing the new one, then restart.
The change marks are distracting	Untick **Mark in the model tree what no export shows yet** in **Preferences > Documentation Export**.